Makine Öğrenmesi

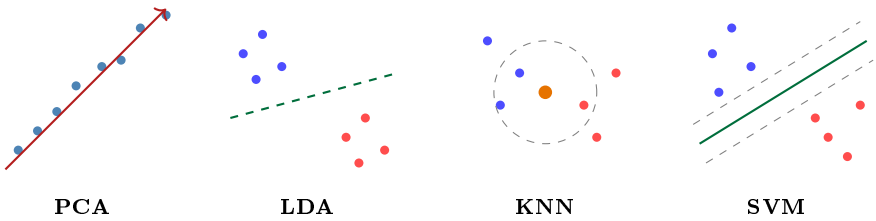
Konu Anlatım Notu

Bölüm 1: Temel Bileşen Analizi (PCA)

Bölüm 2: Lineer Diskriminant Analizi (LDA)

Bölüm 3: K-En Yakın Komşu (KNN)

Bölüm 4: Destek Vektör Makineleri (SVM)



2025–2026 Akademik Yılı

Contents

1 Temel Bileşen Analizi (PCA) 2
1.1 Giriş ve Motivasyon . . . 2
1.2 Matematiksel Temel . . . 2
1.2.1 Adım 1: Veriyi Ortalama . . . 2
1.2.2 Adım 2: Kovaryans Matrisini Hesaplama . . . 2
1.2.3 Adım 3: Özdeğer Ayrışımı . . . 3
1.2.4 Adım 4: Boyut İndirgeme . . . 3
1.3 Açıklanan Varyans Oranı . . . 3
1.4 Geometrik Yorum . . . 4
1.5 Avantajlar ve Sınırlılıklar . . . 4

2 Lineer Diskriminant Analizi (LDA) 6
2.1 Giriş ve Motivasyon . . . 6
2.2 Fisher'ın Lineer Diskriminantı . . . 6
2.2.1 Dağılım Matrisleri . . . 6
2.2.2 Optimal Çözüm . . . 7
2.3 LDA Algoritmasının Adımları . . . 7
2.4 PCA ile LDA Karşılaştırması . . . 7

3 K-En Yakın Komşu (KNN) 8
3.1 Giriş ve Temel Fikir . . . 8
3.2 Uzaklık Metrikleri . . . 8
3.2.1 Öklid Uzaklığı (Euclidean) . . . 8
3.2.2 Manhattan Uzaklığı (L1) . . . 8
3.2.3 Minkowski Uzaklığı (Genelleştirilmiş) . . . 9
3.3 k Değerinin Seçimi . . . 9
3.4 KNN ile Regresyon . . . 9
3.5 Ölçeklemenin Önemi . . . 10
3.6 Avantajlar ve Sınırlılıklar . . . 10

4 Destek Vektör Makineleri (SVM) 11
4.1 Giriş ve Motivasyon . . . 11
4.2 Matematiksel Formülasyon . . . 11
4.2.1 Doğrusal Sınıflandırıcı . . . 11

4.2.2 Hard Margin SVM 11
4.2.3 Margin Kavramı 12
4.3 Destek Vektörleri 12
4.4 Soft Margin SVM 12
4.5 Kernel Trick (Çekirdek Hilesi) 13
4.5.1 Yaygın Kernel Fonksiyonları 13
4.6 RBF Kernel'de γ Parametresi 13
4.7 SVM'in Dual Formülasyonu 13
4.8 Çok Sınıflı SVM 14
4.9 Avantajlar ve Sınırlılıklar 14

5 Genel Karşılaştırma Tablosu 15

1 Temel Bileşen Analizi (PCA)

1.1 Giriş ve Motivasyon

Gerçek dünya veri setleri genellikle çok sayıda öznitelik (boyut) içerir. Örneğin bir görüntü 1000×1000 piksel içeriyorsa her görüntü 1 000 000 boyutlu bir vektördür. Bu denli yüksek boyutluluk hem hesaplama maliyetini artırır hem de *boyut laneti* (curse of dimensionality) sorununa yol açar.

PCA (Principal Component Analysis), yüksek boyutlu veriyi daha düşük boyutlu bir alt uzaya yansıtarak boyut indirgeme yapan *denetimsiz* (unsupervised) bir yöntemdir. Temel fikir, verideki en fazla varyansı koruyan yönleri bulmaktır.

Tanım: Temel Bileşen

Temel bileşenler, veri matrisinin kovaryans matrisinin özvektörleridir. Birinci temel bileşen verideki en yüksek varyans yönünü gösterir; ikinci temel bileşen, birinciye dik olup ikinci en yüksek varyans yönünü verir ve bu böyle devam eder.

1.2 Matematiksel Temel

n örnek ve p öznitelikten oluşan bir veri matrisi $\mathbf{X} \in \mathbb{R}^{n \times p}$ verilsin.

1.2.1 Adım 1: Veriyi Ortalama

Her öznitelikten kendi ortalamasını çıkarın:

$$\tilde{x}_{ij} = x_{ij} - \bar{x}_j, \qquad \bar{x}_j = \frac{1}{n} \sum_{i=1}^{n} x_{ij}$$

Bu işlem verinin merkezini orijine taşır ve kovaryans hesabını sadeleştirir.

1.2.2 Adım 2: Kovaryans Matrisini Hesaplama

Ortalanmış veri $\tilde{\mathbf{X}}$ ile kovaryans matrisi:

$$\mathbf{C} = \frac{1}{n-1} \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \ \in \ \mathbb{R}^{p \times p}$$

Kovaryans matrisinin (i, j) elemanı:

$$C_{ij} = \frac{1}{n-1} \sum_{k=1}^{n} \tilde{x}_{ki} \, \tilde{x}_{kj}$$

Önemli Özellik

Kovaryans matrisi **simetrik** ve **pozitif yarı-tanımlı**dır: $\mathbf{C} = \mathbf{C}^T$ ve tüm özdeğerleri $\lambda_i \geq 0$'dır. Köşegen elemanları $C_{ii} = \mathrm{Var}(x_i)$ olup, köşegen dışı elemanlar kovaryansları verir.

1.2.3 Adım 3: Özdeğer Ayrışımı

Kovaryans matrisinin özdeğer problemi çözülür:

$$\mathbf{C}\mathbf{v}_i = \lambda_i \mathbf{v}_i, \qquad i = 1, 2, \ldots, p$$

Burada λ_i **özdeğer** (eigenvalue), $\mathbf{v}_i$ ise karşılık gelen **özvektör** (eigenvector) olup, $\|\mathbf{v}_i\| = 1$ olacak şekilde normalize edilir.

Karakteristik denklem:

$$\det(\mathbf{C} - \lambda \mathbf{I}) = 0$$

1.2.4 Adım 4: Boyut İndirgeme

Özdeğerler büyükten küçüğe sıralanır: $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0$.

İlk k özvektör seçilerek **yansıtım matrisi** oluşturulur:

$$\mathbf{W} = [\mathbf{v}_1 \;\; \mathbf{v}_2 \;\; \cdots \;\; \mathbf{v}_k] \;\in\; \mathbb{R}^{p \times k}$$

Dönüştürülmüş veri:

$$\mathbf{Z} = \tilde{\mathbf{X}}\,\mathbf{W} \;\in\; \mathbb{R}^{n \times k}$$

1.3 Açıklanan Varyans Oranı

Her bir temel bileşenin açıkladığı varyans oranı:

$$\mathrm{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^{p} \lambda_j}$$

Kumülatif açıklanan varyans:

$$\mathrm{CEVR}(k) = \sum_{i=1}^{k} \mathrm{EVR}_i$$

Genellikle $\mathrm{CEVR}(k) \geq \%90$ veya $\%95$ olacak şekilde k seçilir.

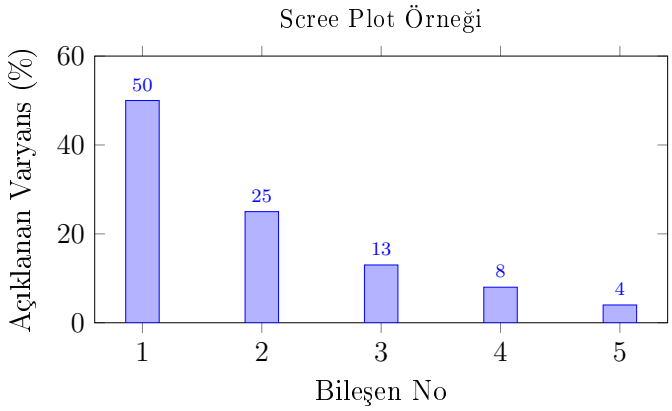


Figure 1: Scree plot: bileşen sayısına göre açıklanan varyans yüzdesi. Dirsek noktası (elbow) k seçimi için kullanılır.

1.4 Geometrik Yorum

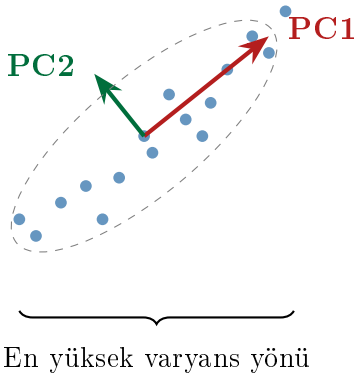


Figure 2: PCA, veri bulutunun en çok yayıldığı yönleri (temel bileşenleri) bulur.

PCA geometrik olarak, veri bulutunu çevreleyen elipsin eksenlerini bulmaya karşılık gelir. En uzun eksen (en büyük özdeğerli yön) birinci temel bileşendir.

1.5 Avantajlar ve Sınırlılıklar

Avantajlar

- Boyut indirgeme ile hesaplama maliyetini azaltır.
- Gürültüyü (noise) filtreleyerek veri kalitesini artırır.
- Görselleştirme için 2D/3D uzaya indirgeme sağlar.
- Çoklu bağlantı (multicollinearity) problemini giderir.
- Denetimsizdir – etiket bilgisi gerektirmez.

Sınırlılıklar

- Yalnızca **doğrusal** ilişkileri yakalar (doğrusal olmayan yapılar için Kernel PCA gerekir).
- Ölçeklemeye duyarlıdır – öncesinde **standardizasyon** gerekir.
- Temel bileşenlerin **yorumlanabilirliği** düşüktür.
- Sınıf bilgisini dikkate almaz (sınıflandırmada LDA tercih edilebilir).

2 Lineer Diskriminant Analizi (LDA)

2.1 Giriş ve Motivasyon

PCA verideki toplam varyansı en çok koruyan yönleri bulurken, **LDA (Linear Discriminant Analysis)** sınıfları en iyi **ayıran** yönleri bulmayı amaçlar. Bu nedenle LDA bir *denetimli* (supervised) boyut indirgeme yöntemidir.

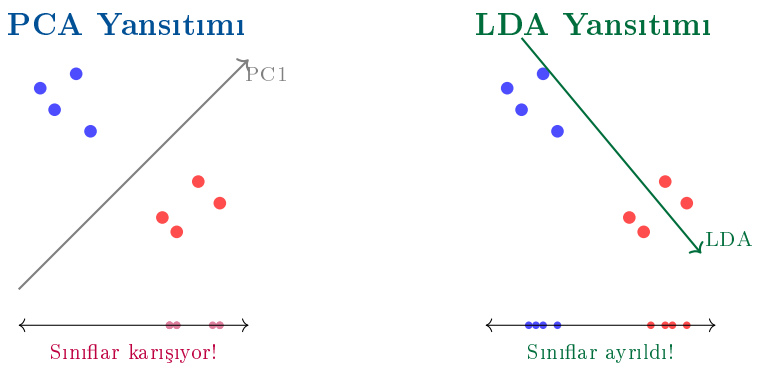


Figure 3: PCA ile LDA'nın karşılaştırılması: PCA toplam varyansı, LDA sınıf ayrımını en çok yapar.

2.2 Fisher'ın Lineer Diskriminantı

İki sınıflı problem için Fisher, aşağıdaki oranı en büyük yapan $\mathbf{w}$ yönünü arar:

Fisher Kriteri

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

Bu oran, **sınıflar arası dağılımın** sınıf içi dağılıma olan oranıdır. $J(\mathbf{w})$ ne kadar büyükse sınıflar o kadar iyi ayrılır.

2.2.1 Dağılım Matrisleri

Sınıf içi dağılım matrisi (within-class scatter):

$$\mathbf{S}_W = \sum_{k=1}^{K} \mathbf{S}_k, \qquad \mathbf{S}_k = \sum_{\mathbf{x}_i \in C_k} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^T$$

Sınıflar arası dağılım matrisi (between-class scatter):

$$\mathbf{S}_B = \sum_{k=1}^{K} n_k (\boldsymbol{\mu}_k - \boldsymbol{\mu})(\boldsymbol{\mu}_k - \boldsymbol{\mu})^T$$

Burada $\boldsymbol{\mu}_k$ k-inci sınıfın ortalaması, $\boldsymbol{\mu}$ tüm verinin ortalaması, n_k ise sınıftaki örnek sayısıdır.

2.2.2 Optimal Çözüm

İki sınıflı durumda optimal yön doğrudan hesaplanabilir:

$$\boxed{\mathbf{w}^* = \mathbf{S}_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)}$$

Çok sınıflı ($K > 2$) durumda ise genelleştirilmiş özdeğer problemi çözülür:

$$\mathbf{S}_W^{-1}\mathbf{S}_B\,\mathbf{v} = \lambda\,\mathbf{v}$$

En fazla $K-1$ adet sıfırdan farklı özdeğer elde edilir; dolayısıyla LDA en fazla $K-1$ boyuta indirgeme yapar.

2.3 LDA Algoritmasının Adımları

1. Her sınıf için ortalama vektörü $\boldsymbol{\mu}_k$ hesapla.
2. Sınıf içi dağılım matrisini $\mathbf{S}_W$ hesapla.
3. Sınıflar arası dağılım matrisini $\mathbf{S}_B$ hesapla.
4. $\mathbf{S}_W^{-1}\mathbf{S}_B$ matrisinin özvektörlerini bul.
5. En büyük $k \leq K-1$ özdeğere karşılık gelen özvektörleri seç.
6. Veriyi bu özvektörlerle yansıt.

2.4 PCA ile LDA Karşılaştırması

Özellik	**PCA**	**LDA**
Tür	Denetimsiz	Denetimli
Amaç	Varyansı en çoklama	Sınıf ayrımını en çoklama
Çıktı boyutu	1 ile p arası	En fazla $K-1$
Kullanım	Keşifsel analiz, ön işleme	Sınıflandırma öncesi boyut indirgeme
Etiket gerekir mi?	Hayır	Evet

Dikkat Edilmesi Gerekenler

- $\mathbf{S}_W$ tekil (singular) olabilir – bu durumda regularizasyon veya PCA ön işleme uygulanır.
- LDA, sınıfların **normal dağılımlı** ve **eşit kovaryanslı** olduğunu varsayar.
- Küçük örneklem sayılarında $\mathbf{S}_W$ tahmini güvenilir olmayabilir.

3 K-En Yakın Komşu (KNN)

3.1 Giriş ve Temel Fikir

KNN, en basit ve sezgisel makine öğrenmesi algoritmalarından biridir. Eğitim aşamasında herhangi bir model öğrenmez; bunun yerine tüm eğitim verisini hafızada tutar. Bu nedenle *tembel öğrenme* (lazy learning) veya *örnek tabanlı öğrenme* (instance-based learning) olarak adlandırılır.

KNN Karar Kuralı

Yeni bir $\mathbf{x}$ noktasını sınıflandırmak için:

1. Eğitim setindeki tüm noktalara olan uzaklıkları hesapla.
2. En yakın k komşuyu seç.
3. Bu k komşunun sınıf etiketleri arasında çoğunluk oyu al.
4. En çok oy alan sınıfı $\mathbf{x}$'e ata.

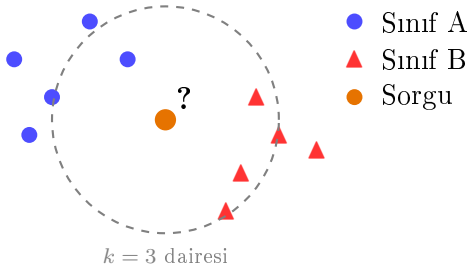


Figure 4: KNN: sorgu noktası etrafındaki $k = 3$ en yakın komşuya bakılarak sınıflandırma yapılır.

3.2 Uzaklık Metrikleri

KNN'in performansı kullanılan uzaklık metriğine bağlıdır. En yaygın metrikler:

3.2.1 Öklid Uzaklığı (Euclidean)

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{p} (x_i - y_i)^2} = \|\mathbf{x} - \mathbf{y}\|_2$$

En sık kullanılan metriktir; sürekli ve eşit ölçekli öznitelikler için uygundur.

3.2.2 Manhattan Uzaklığı (L1)

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{p} |x_i - y_i| = \|\mathbf{x} - \mathbf{y}\|_1$$

Yüksek boyutlu verilerde ve aykırı değerlere karşı Öklid'e göre daha dayanıklıdır.

3.2.3 Minkowski Uzaklığı (Genelleştirilmiş)

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{p} |x_i - y_i|^q \right)^{1/q}$$

$q = 1$ Manhattan, $q = 2$ Öklid uzaklığını verir.

3.3 k Değerinin Seçimi

k değeri algoritmanın en kritik hiperparametresidir:

	Küçük k	**Büyük k**
Bias	Düşük (esnek model)	Yüksek (katı model)
Varyans	Yüksek (gürültüye duyarlı)	Düşük (stabil)
Karar sınırı	Karmaşık, düzensiz	Pürüzsüz, basit
Risk	Aşırı uyum (overfitting)	Eksik uyum (underfitting)

k Seçimi İçin Pratik Kurallar

- Genellikle $k = \sqrt{n}$ (örnek sayısının karekökü) iyi bir başlangıçtır.
- İkili sınıflandırmada beşitliği önlemek için k **tek sayı** seçilir.
- En iyi yöntem: **çapraz doğrulama** (cross-validation) ile farklı k değerlerini denemek.

3.4 KNN ile Regresyon

KNN yalnızca sınıflandırma için değil, regresyon için de kullanılabilir. Çoğunluk oyu yerine k komşunun hedef değerlerinin **ortalaması** alınır:

$$\hat{y}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i$$

Ağırlıklı KNN'de yakın komşulara daha fazla ağırlık verilebilir:

$$\hat{y}(\mathbf{x}) = \frac{\sum_{i \in N_k(\mathbf{x})} w_i \, y_i}{\sum_{i \in N_k(\mathbf{x})} w_i}, \qquad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)^2}$$

3.5 Ölçeklemenin Önemi

KNN uzaklık tabanlı olduğu için özniteliklerin ölçeği sonuçları doğrudan etkiler. Örneğin "yaş (0–100)" ve "gelir (0–1 000 000)" birlikte kullanılırsa, gelir uzaklığa çok daha fazla katkı yapar.

Çözüm: Öznitelikleri normalize veya standartlaştırın:

$$z = \frac{x - \mu}{\sigma} \qquad \text{(Z-skor standardizasyonu)}$$

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \qquad \text{(Min-Max normalizasyonu)}$$

3.6 Avantajlar ve Sınırlılıklar

Avantajlar

- Uygulanması çok kolay ve sezgiseldir.
- Eğitim süresi yoktur (tembel öğrenme).
- Doğrusal olmayan karar sınırları oluşturabilir.
- Çok sınıflı problemlerde doğal olarak çalışır.
- Hem sınıflandırma hem regresyona uygulanabilir.

Sınırlılıklar

- Tahmin zamanı yavaştır: her tahmin için tüm eğitim verisiyle uzaklık hesaplanır.
- Bellek tüketimi yüksektir (tüm veri saklanır).
- Yüksek boyutlulukta performans düşer (*boyut laneti*).
- Ölçeklemeye çok duyarlıdır.
- Dengesiz veri setlerinde çoğunluk sınıfına yönelir.

4 Destek Vektör Makineleri (SVM)

4.1 Giriş ve Motivasyon

SVM (Support Vector Machine), özellikle **iki sınıflı sınıflandırma** problemlerinde güçlü performans gösteren bir *denetimli* öğrenme algoritmasıdır. Temel fikir, iki sınıfı birbirinden **en geniş margin** (boşluk) ile ayıran **optimal hiper-düzlemi** bulmaktır.

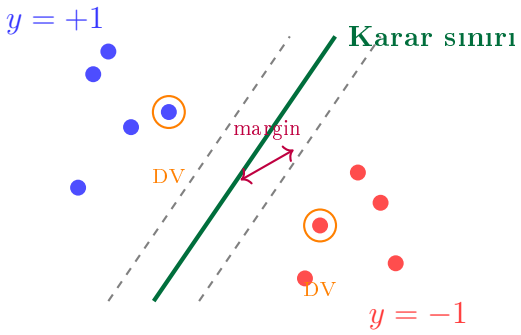


Figure 5: SVM: karar sınırı, margin bölgesi ve destek vektörleri (DV). Yalnızca destek vektörleri karar sınırını belirler.

4.2 Matematiksel Formülasyon

4.2.1 Doğrusal Sınıflandırıcı

Karar fonksiyonu:

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$$

Sınıflandırma kuralı:

$$\hat{y} = \text{sign}(f(\mathbf{x})) = \begin{cases} +1 & \text{eğer } \mathbf{w}^T\mathbf{x} + b > 0 \\ -1 & \text{eğer } \mathbf{w}^T\mathbf{x} + b < 0 \end{cases}$$

4.2.2 Hard Margin SVM

Tüm noktalar doğru sınıflandırılıyorsa (**doğrusal olarak ayrılabilir** veri):

Hard Margin Optimizasyonu

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2$$

$$\text{kısıt:} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \qquad \forall\, i = 1, \ldots, n$$

Bu problem $\|\mathbf{w}\|$'yi küçültmek, yani margin'i $\dfrac{2}{\|\mathbf{w}\|}$ büyütmek anlamına gelir.

4.2.3 Margin Kavramı

Bir $\mathbf{x}_i$ noktasının hiper-düzleme olan uzaklığı:

$$\text{uzaklık} = \frac{|\mathbf{w}^T\mathbf{x}_i + b|}{\|\mathbf{w}\|}$$

Fonksiyonel margin: $\hat{\gamma}_i = y_i(\mathbf{w}^T\mathbf{x}_i + b)$

Geometrik margin: $\gamma_i = \dfrac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|}$

Toplam margin genişliği $= \dfrac{2}{\|\mathbf{w}\|}$ (iki destek vektörü arasındaki mesafe).

4.3 Destek Vektörleri

Destek Vektörleri Nedir?

Destek vektörleri, margin sınırları üzerinde bulunan, yani $y_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$ koşulunu sağlayan eğitim örnekleridir. Bunlar karar sınırına en yakın noktalardır ve yalnızca bunlar modeli belirler.

Önemli özellik: Destek vektörleri dışındaki noktaları kaldırmak veya hareket ettirmek karar sınırını değiştirmez. Bu SVM'e güçlü bir genelleme kapasitesi verir.

4.4 Soft Margin SVM

Gerçek verilerde sınıflar genellikle mükemmel doğrusal olarak ayrılamaz. **Soft margin** SVM, bazı noktaların margin'ı ihlal etmesine izin verir:

$$\min_{\mathbf{w},b,\xi} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

$$\text{kısıtlar:} \qquad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0, \qquad \forall\, i$$

Burada ξ_i **gevşeme değişkenleri** (slack variables) olup her noktanın margin ihlalini ölçer; $C > 0$ ise **ceza parametresi**dir.

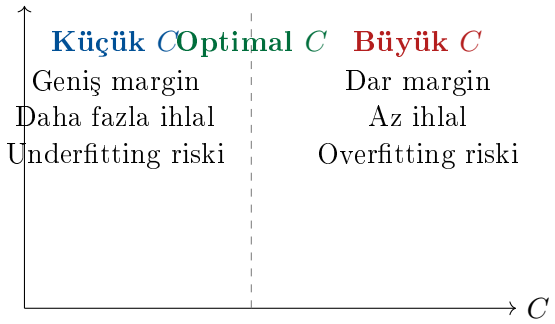


Figure 6: C parametresinin etkisi: bias–varyans dengesi.

4.5 Kernel Trick (Çekirdek Hilesi)

Doğrusal olarak ayrılamayan veriler için SVM, veriyi daha yüksek boyutlu bir uzaya taşır. **Kernel trick** sayesinde bu dönüşüm açıkça yapılmaz; bunun yerine çekirdek fonksiyonu kullanılır:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

4.5.1 Yaygın Kernel Fonksiyonları

Kernel	Formül
Doğrusal	$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
Polinom	$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma\, \mathbf{x}_i^T \mathbf{x}_j + r)^d$
RBF (Gauss)	$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\Big(-\gamma \lVert \mathbf{x}_i - \mathbf{x}_j \rVert^2\Big)$
Sigmoid	$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma\, \mathbf{x}_i^T \mathbf{x}_j + r)$

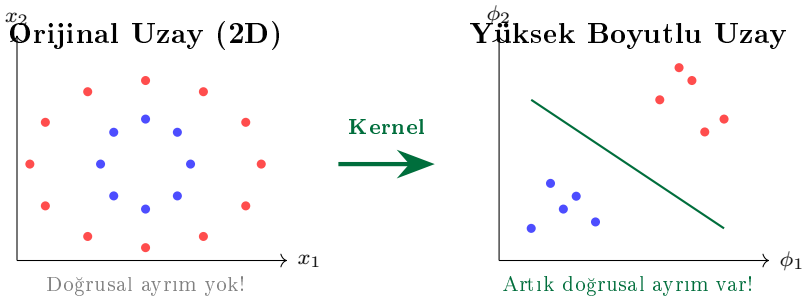


Figure 7: Kernel trick: orijinal uzayda doğrusal olarak ayrılamayan veri, yüksek boyutlu uzayda ayrılabilir hale gelir.

4.6 RBF Kernel'de γ Parametresi

RBF kernel'de γ parametresi karar sınırının esnekliğini kontrol eder:

	Küçük γ	**Büyük γ**
Etki alanı	Her nokta geniş etki alanına sahip	Her nokta dar etki alanına sahip
Karar sınırı	Pürüzsüz, düz	Karmaşık, kıvrımlı
Risk	Underfitting	Overfitting

4.7 SVM'in Dual Formülasyonu

Lagrange çarpanları $\alpha_i \geq 0$ kullanılarak dual problem:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

$$\text{kısıtlar:} \quad \sum_{i=1}^{n} \alpha_i y_i = 0, \qquad 0 \le \alpha_i \le C$$

Önemli: $\alpha_i > 0$ olan noktalar **destek vektörleri**dir. Diğer noktaların $\alpha_i = 0$ olması, onların karar sınırına katkı yapmadığı anlamına gelir.

Tahmin:

$$f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

4.8 Çok Sınıflı SVM

SVM doğal olarak iki sınıflıdır. Çok sınıflı problemler için:

Bire–Hepsine (One-vs-Rest, OVR): K sınıf için K ayrı SVM eğitilir. Her SVM bir sınıfı diğerlerinden ayırır. Tahmin: en yüksek $f(\mathbf{x})$ veren sınıf seçilir.

Bire–Bir (One-vs-One, OVO): $\binom{K}{2}$ adet ikili SVM eğitilir. Tahmin: çoğunluk oyu ile belirlenir.

4.9 Avantajlar ve Sınırlılıklar

Avantajlar

- Yüksek boyutlu verilerde etkilidir ($p > n$ durumunda bile).
- Kernel trick ile doğrusal olmayan sınırlar öğrenilebilir.
- Genelleme performansı yüksektir (margin maksimizasyonu).
- Yalnızca destek vektörlerine bağlıdır – bellek verimlidir.
- Överi uyuma karşı dayanıklıdır (C ile kontrol).

Sınırlılıklar

- Büyük veri setlerinde eğitim süresi uzundur ($O(n^2)$ – $O(n^3)$).
- Kernel ve hiperparametre (C, γ) seçimi kritiktir.
- Olasılık çıktısı vermez (ancak Platt ölçekleme kullanılabilir).
- Çok sınıflı problemlerde doğrudan destek yoktur.
- Gürültülü verilerde (çakışan sınıflar) performans düşebilir.

5 Genel Karşılaştırma Tablosu

Özellik	**PCA**	**LDA**	**KNN**	**SVM**
Tür	Denetimsiz	Denetimli	Denetimli	Denetimli
Görev	Boyut indirgeme	Boyut indirgeme + sınıflandırma	Sınıflandırma / Regresyon	Sınıflandırma / Regresyon
Model tipi	Doğrusal dönüşüm	Doğrusal dönüşüm	Örnek tabanlı (tembel)	Margin tabanlı
Anahtar kavram	Varyans en çoklama	S_B/S_W en çoklama	Uzaklık ölçüsü ve k	Margin en çoklama
Hiperparametre	k (bileşen sayısı)	$k \leq K-1$	k, uzaklık metriği	C, kernel, γ
Ölçekleme	Gerekli	Gerekli	Çok gerekli	Gerekli
Eğitim hızı	Hızlı	Hızlı	Yok (tembel)	Yavaş (n büyükse)
Tahmin hızı	Hızlı	Hızlı	Yavaş	Hızlı